

Hostel Management System

Submitted for
Data Engineering(UCS667)
End Semester Evaluation

Submitted by:

Akshat Goyal (102303475)

BE Third Year COE
Sub Group: 3C34

Submitted to:
Mrs Archana Singh



Computer Science and Engineering Department
Thapar Institute of Engineering & Technology, Patiala

January - May 2026

Sr no.	Title	Page no.
1.	Introduction	2
2.	Problem Statement	3
3.	Specific Requirements	4-5
4.	Context Level Diagram And DFD	6
5.	System Specification	7-9
6.	Tools Used	9
7.	Sample Screenshots	10-13
8.	Output Reports	14
9.	Conclusion	15

1. INTRODUCTION

The increasing number of students residing in hostels within universities and institutions has created a significant demand for efficient management systems. Traditional hostel management processes often rely on manual record-keeping or fragmented digital tools, making it difficult to manage student data, room allocation, complaints, and fee records effectively.

Students frequently face issues such as delayed complaint resolution, lack of transparency in room allocation, and difficulty tracking their fee payments. On the administrative side, managing large volumes of student records, monitoring room occupancy, handling complaints, and maintaining fee data becomes complex and error-prone.

To address these challenges, the Hostel Management System has been developed as a full-stack web application. The system provides an integrated platform for administrators to manage hostel operations efficiently, while also offering student-oriented functionalities such as complaint submission and fee tracking.

The application is built using Node.js and Express.js for backend processing, MongoDB with Mongoose for database management, and a single-page dashboard interface using HTML, CSS, and JavaScript for frontend interaction.

The system allows:

- Centralized management of students, rooms, complaints, and fees
- Dynamic filtering and sorting of data
- Automated room allocation logic
- Complaint tracking and resolution
- Fee payment tracking with detailed history

This project demonstrates practical implementation of NoSQL database concepts, RESTful APIs, and frontend-backend integration in a real-world hostel management scenario.

2. PROBLEM STATEMENT

Managing hostel operations manually or using disconnected systems leads to inefficiencies and errors. Administrators face challenges in tracking room occupancy, managing student data, resolving complaints, and maintaining accurate fee records.

Students also experience issues such as:

- Lack of visibility into room allocation status
- Delayed complaint resolution
- Difficulty tracking fee payments

Existing systems often lack:

- Integrated data management
- Real-time updates
- User-friendly interfaces
- Automated workflows

Therefore, there is a need for a centralized, efficient, and user-friendly Hostel Management System that simplifies administrative operations, ensures data consistency, and enhances transparency for both administrators and students.

3. SPECIFIC REQUIREMENTS

A) Functional Requirements

1. **Dashboard Metrics**

- Display total students
- Show students without rooms
- Show students with pending fees
- Display available rooms and capacity
- Show pending complaints

2. **Student Management**

- Add student records
- Filter students by multiple criteria
- Sort students based on ID or year
- Display room assignment and allocation status

3. **Room Management**

- View all rooms with details
- Filter rooms (All / Available / Full)
- Allocate rooms to eligible students
- Remove room assignments

4. **Complaint Management**

- View all complaints
- Filter complaints by status
- Resolve complaints
- Track complaint history

5. **Fee Management**

- View fee records of all students
- Filter by payment status
- Track payments and balances
- Display payment history

6. Student Complaint System

- Search student by ID or name
- Add and delete complaints

7. Student Fee System

- View fee summary
- Pay fees using different methods
- Track payment history with filters and sorting

8. Student Panel

- Validate student credentials
 - Display student details, allocation, complaints, and activity logs
-

B) Non-Functional Requirements

1. Performance

Efficient handling of database queries and fast response time

2. Scalability

Ability to handle increasing number of students and records

3. Reliability

Accurate data processing and consistency

4. Security

Input validation and controlled data access

5. Maintainability

Modular code structure for easy updates

6. Usability

Simple and intuitive user interface

7. Availability

System should be accessible whenever required

4. CONTEXT LEVEL DIAGRAM AND DFD

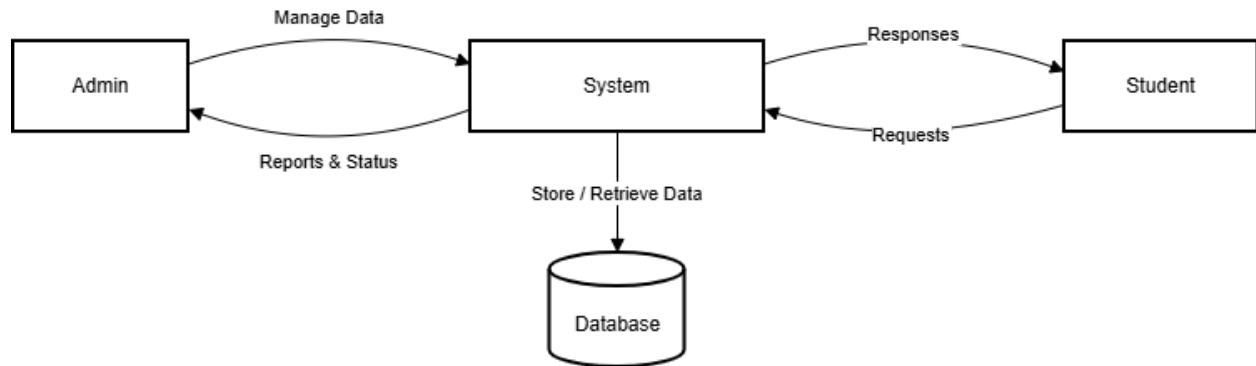


Fig: Context Level Diagram of Hostel Management System

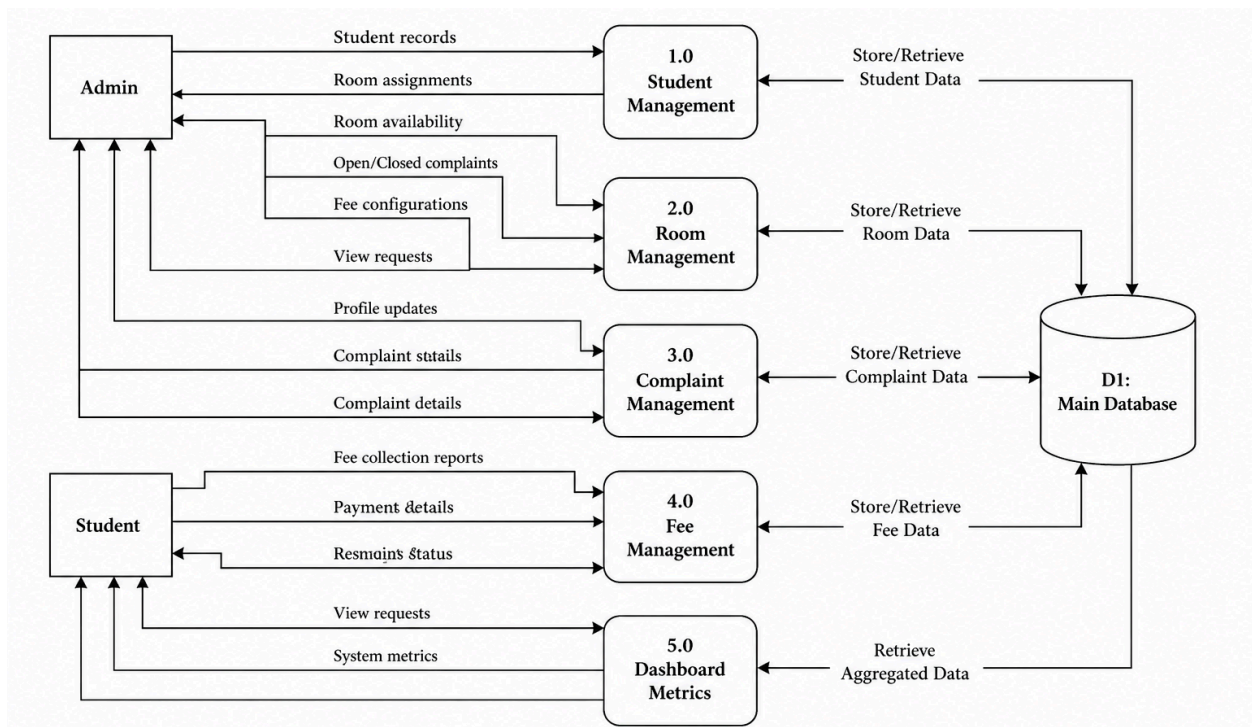


Fig: Level 1 Data Flow Diagram of CourseHub System

5. SYSTEM SPECIFICATION

The Hostel Management System is developed using modern web technologies to ensure efficient data handling, smooth execution, and an interactive user experience. The system follows a client-server architecture where the frontend communicates with backend APIs, and the backend interacts with the MongoDB database for storing and retrieving data.

1. Hardware Specifications

Development Environment:

- Processor: Intel Core i3/i5/i7 or AMD equivalent
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: Minimum 20 GB free space
- Display: Full HD Monitor
- Internet Connection: Required for installation and updates

2. Software Specifications

Operating System:

- Windows 10/11
- macOS
- Linux

Frontend Technologies:

- HTML for structure
- CSS for styling
- JavaScript for dynamic behavior and single-page dashboard rendering

Backend Technologies:

- Node.js for server-side execution
- Express.js for API handling and routing

Database:

- MongoDB for storing hostel-related data
- Mongoose for schema definition and database interaction

Development Tools:

- Visual Studio Code for coding
- Postman for API testing
- MongoDB Compass for database visualization

3. System Architecture

The Hostel Management System follows a three-tier architecture:

Presentation Layer:

The frontend dashboard allows administrators and users to interact with the system through a browser-based interface. It dynamically renders data without requiring page reloads.

Application Layer:

The backend developed using Node.js and Express.js processes requests, executes business logic, and handles communication between the frontend and database.

Database Layer:

MongoDB stores all system data, including student records, room details, complaints, and fee transactions.

4. Database Schema Fields

The database is structured into multiple collections, each representing a key module of the system.

Student Collection:

- studentId (unique numeric identifier)
- name, department, year, phone
- roomId (reference to Room)
- allocation details (status, appliedAt, allocatedAt)
- skills
- internship details
- complaints (embedded documents)
- activityLog (embedded documents)

Room Collection:

- roomNumber
- hostelBlock
- capacity
- occupants (array of student references)

Fee Collection:

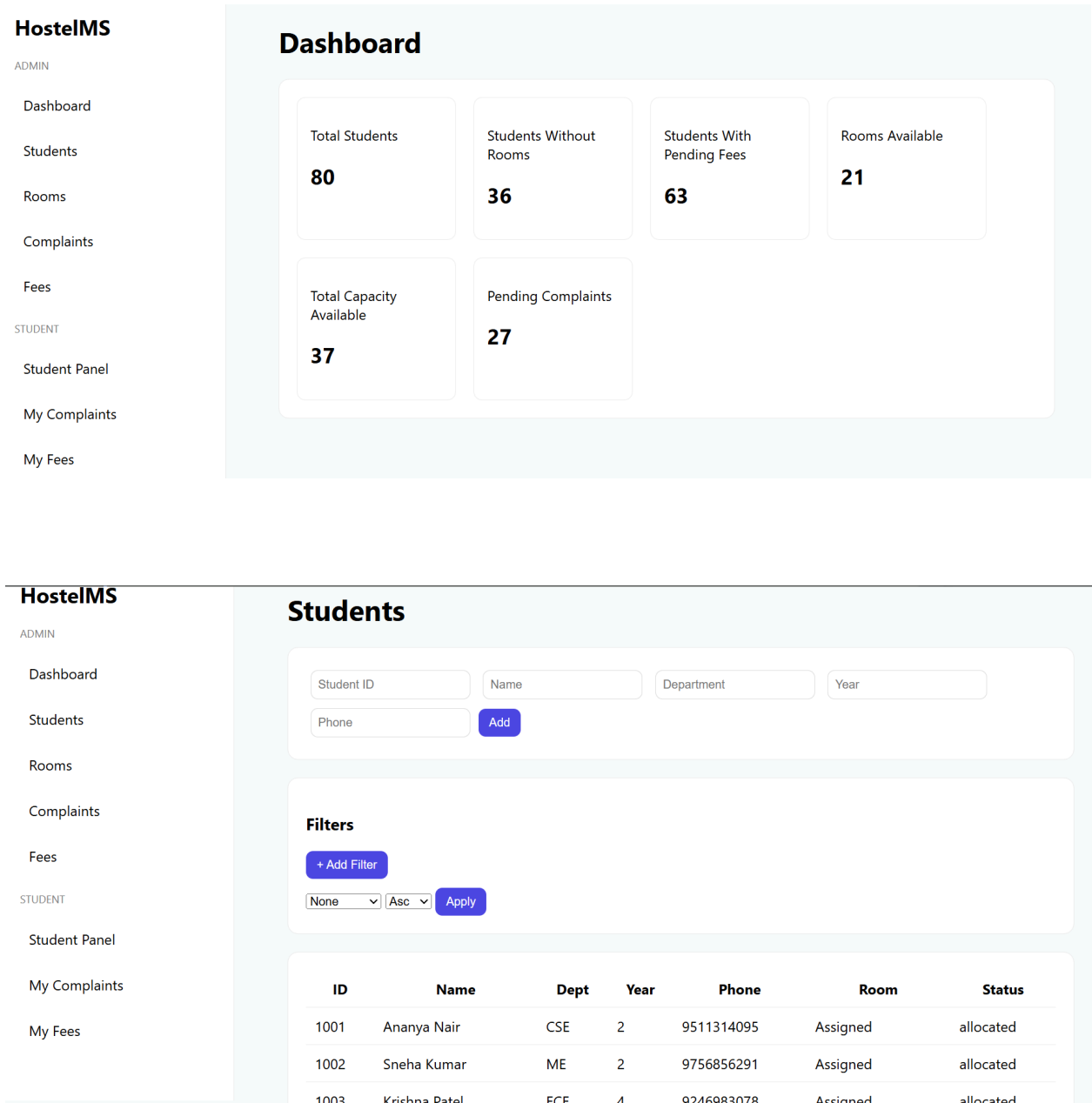
- studentId (reference to Student)
- totalFees
- payments (amount, date, method)
- status (unpaid, partial, paid)

6. TOOLS USED

The development of the Hostel Management System required various tools and technologies for frontend development, backend processing, database management, and testing.

- **MongoDB**
MongoDB is used as a NoSQL database to store and manage student, room, complaint, and fee-related data efficiently.
- **Express.js**
Express.js is used as the backend framework to build REST APIs and handle routing logic.
- **Node.js**
Node.js serves as the runtime environment for executing backend JavaScript code.
- **HTML, CSS, JavaScript**
These technologies are used for developing the frontend dashboard and creating a dynamic user interface.
- **Mongoose**
Mongoose is used as an Object Data Modeling (ODM) library to define schemas and interact with MongoDB.
- **MongoDB Compass**
MongoDB Compass is used for visualizing database collections and inspecting stored data.
- **Postman**
Postman is used for testing API endpoints and verifying backend functionality.
- **Visual Studio Code**
Visual Studio Code is used as the primary development environment for writing and managing code.

7. SAMPLE SCREENSHOTS



HostelIMS

ADMIN

Dashboard

Students

Rooms

Complaints

Fees

STUDENT

Student Panel

My Complaints

My Fees

Rooms

Allocate Room

Only pending students and available rooms are listed.

Show:

Room	Block	Capacity	Occupied	Status
101	C	2	0	Available
102	B	4	3	Available
103	B	3	3	Full
104	A	3	1	Available

HostelIMS

ADMIN

Dashboard

Students

Rooms

Complaints

Fees

STUDENT

Student Panel

My Complaints

My Fees

Complaints

Show:

Student ID	Student	Title	Status	Date	Action
1001	Ananya Nair	Study Table Issue	pending	4/30/2026	Resolve
1005	Kavya Kumar	Fan not working	pending	3/16/2026	Resolve
1010	Riya Gupta	Water leakage	resolved	4/9/2026	-
1010	Riya Gupta	Light not working	pending	3/26/2026	Resolve
1012	Isha Patel	Water leakage	pending	4/11/2026	Resolve
1014	Sai Yadav	Light not working	pending	3/7/2026	Resolve
1015	Meera Yadav	Light not working	resolved	3/15/2026	-

HostelMS

ADMIN

Dashboard

Students

Rooms

Complaints

Fees

STUDENT

Student Panel

My Complaints

My Fees

Fees

Show: All Apply

Student	Total Fees	Paid	Status
1001 - Ananya Nair	55000	55000	paid
1002 - Sneha Kumar	45000	0	unpaid
1003 - Krishna Patel	55000	0	unpaid
1004 - Neha Verma	45000	0	unpaid
1005 - Kavya Kumar	45000	0	unpaid
1006 - Aarav Yadav	50000	27500	partial
1007 - Pooja Reddy	55000	55000	paid
1008 - Rohan Sharma	55000	27500	partial

HostelMS

ADMIN

Dashboard

Students

Rooms

Complaints

Fees

STUDENT

Student Panel

My Complaints

My Fees

Student Panel

1004

Neha Verma

Load

Student Details

Student ID: 1004**Name:** Neha Verma**Department:** ME**Year:** 1**Phone:** 9686261269**Skills:** JavaScript, MongoDB**Internship:** -**Room Allocation:** Not Assigned**Allocation Status:** pending**Applied At:** 4/5/2026, 8:31:41 PM**Allocated At:** -**Created At:** 4/30/2026, 7:12:14 PM**Updated At:** 4/30/2026, 7:21:02 PM

Complaints

No complaints found.

Activity Log (latest at bottom)

Type	Description	Date
registration	Student registered	4/5/2026, 8:31:41 PM

HostelMS

ADMIN

Dashboard

Students

Rooms

Complaints

Fees

STUDENT

Student Panel

My Complaints

My Fees

My Complaints

Search by Student ID ▾

1004

Search

Only one student can be selected at a time.

Search Result (Single Select)

1004 - Neha Verma ▾

Add Complaint

Complaint title

Add Complaint

HostelMS

ADMIN

Dashboard

Students

Rooms

Complaints

Fees

STUDENT

Student Panel

My Complaints

My Fees

My Fees

Search by Name ▾

Neha Verma

Search

Select one student to manage fee details.

Search Result (Single Select)

1004 - Neha Verma ▾

Fee Summary

Student: 1004 - Neha Verma**Total Fees:** 45000**Total Paid:** 0**Remaining:** 45000**Status:** unpaid

Pay Fees

Amount

Cash ▾

Pay Now

Payment History

Method: All ▾ Sort By: Date ▾ Desc ▾

Date

Amount

Method

No payment history to display

8. OUTPUT REPORTS

This section highlights the outputs generated during the execution of the Hostel Management System.

a) Dashboard Output

The system displays real-time statistics including total students, available rooms, pending complaints, and fee summaries.

b) Student Management Output

The system allows efficient retrieval, filtering, and sorting of student records based on multiple criteria.

c) Room Management Output

The system displays room availability, occupancy status, and successful allocation of rooms to students.

d) Complaint Management Output

The system shows complaint records, supports filtering by status, and allows updating complaint resolution.

e) Fee Management Output

The system provides detailed fee records including total fees, payment history, remaining balance, and status.

f) API Testing Output

All API endpoints were tested using Postman to ensure proper functionality and accurate responses.

g) Final System Output

The final output is a fully functional hostel management system integrating all modules seamlessly.

9. CONCLUSION

The Hostel Management System successfully demonstrates the development of a full-stack web application for managing hostel operations efficiently. The system integrates student management, room allocation, complaint handling, and fee tracking into a unified platform.

The project provided practical experience in backend development using Node.js and Express.js, database design using MongoDB, and frontend development using HTML, CSS, and JavaScript. It also highlighted the importance of modular architecture, API design, and efficient data handling.

The system improves operational efficiency, reduces manual errors, and provides real-time insights into hostel activities. It ensures better transparency and ease of management for administrators while offering useful functionalities for students.

In the future, the system can be enhanced by adding features such as authentication, role-based access control, notification systems, and advanced analytics dashboards.

Overall, the project serves as a practical implementation of modern web technologies applied to solve real-world hostel management problems.